

Summary Report: NumPy-II Array Operations and Statistics

This report summarises advanced NumPy operations, statistical functions, and alternative array creation methods based on the provided sources.

1. Structural Operations on Arrays

NumPy allows for the modification of array shapes and orientations without necessarily changing the underlying data.

- **Transpose:** This operation turns rows into columns and vice versa, similar to mathematical matrices. You can use the `transpose()` method or the shorthand `.T` attribute. Crucially, transposing returns a new array; the original remains unchanged.
 - **Example:** For a 2x3 matrix, `arr.transpose()` or `arr.T` results in a 3x2 matrix.
 - **Reshaping:** The `reshape()` method changes dimensions (e.g., 1D to 2D) as long as the total number of elements stays the same. You can use `-1` in the new shape to have NumPy automatically calculate the size of one dimension. If the total number of elements does not match the new shape, a `ValueError` is raised.
 - **Concatenating:** The `np.concatenate()` function joins multiple arrays along an existing axis. Arrays must have the identical shape except in the dimension corresponding to the axis of concatenation; otherwise, a `ValueError` occurs.
 - **Splitting:** Using `np.split()`, an array can be broken into multiple equal-sized sub-arrays along a specified axis.
-

2. Sorting Arrays

Sorting arranges elements in hierarchical order, defaulting to ascending order.

- `np.sort()`: Returns a **sorted copy** of the original array.
 - `arr.sort()`: Performs an **in-place sort**, modifying the original array directly.
 - `np.argsort()`: Returns the **indices** that would sort the array, which is useful for indirect sorting.
 - **Axis Logic:** Sorting can be specified by axis:
 - **Axis 0 (Rows):** Operates vertically down the columns.
 - **Axis 1 / Axis -1 (Columns):** Operates horizontally across the rows.
-

3. Statistical Operations

NumPy provides built-in functions to perform statistical analysis across an entire array or along specific axes.

Operation	Description	Source
-----------	-------------	--------

Mean	Calculates the arithmetic mean.
Median	Computes the middle value.
Standard Deviation	Measures data spread (<code>np.std()</code>).
Min/Max	Finds the lowest and highest values.
Sum	Calculates the total of all elements.

4. Python List vs. NumPy Array

The sources highlight several critical differences between these two data structures:

- **Data Type:** Python lists can store mixed data types, while NumPy arrays must contain elements of the same type.
 - **Element-wise Operations:** Arrays support direct operations (e.g., `arr / 3`), whereas lists require loops or comprehensions to perform math on each element.
 - **Functionality:** NumPy provides extensive mathematical functions and library support specifically for scientific computing.
-

5. Advanced Array Creation Methods

Beyond simple lists, NumPy offers specialised functions to generate arrays:

- `np.zeros()` / `np.ones()`: Create arrays filled with 0.0 or 1.0 (float by default).
 - `np.full(shape, fill_value)`: Creates an array where every element is the specified `fill_value`.
 - `np.eye(N, M, k)`: Generates a 2D identity-like matrix with 1s on the k-th diagonal and 0s elsewhere.
 - `np.arange(start, stop, step)`: Similar to Python's `range()`, creating a 1D array of evenly spaced values.
 - `np.random.random(shape)`: Fills an array with random float values between 0 and 1.
 - `np.linspace(start, stop, num)`: Generates a precise number of evenly spaced points between two values.
-

Analogy for Understanding Axes: Imagine a **chest of drawers**. **Axis 0** is like looking at the drawers from the top down; operations affect the vertical stack (the columns). **Axis 1** is like pulling out a single drawer and looking at the items inside from left to right; operations affect the horizontal contents (the rows).